

Algorithms and Data Structures

17.05.2023

Exercise 1 (1.5 points)

Given the following array of integer values, sort it in descending order using the merge sort procedure.

8 5 6 7 13 3 12 2 17 11

Display the content of the array just before the last (and conclusive) merge step (the one delivering the final and sorted array). Please, show the entire content of the array as a sequence of integer values separated by a single space. No other symbols must be included in the response.

Solution

```
### Merge sort
0 1 2 3 4 5 6 7 8 9
8 5 6 7 13 3 12 2 17 11
8 5 6 7 13 - 3 12 2 17 11
8 5 6 - 7 13 - 3 12 2 - 17 11
8 5 - 6 - 7 - 13 - 3 12 - 2 - 17 - 11
8 - 5 - 13 7 - 3 - 12 - 17 11
8 5 - 12 3
8 6 5 - 12 3 2
13 8 7 6 5 - 17 12 11 3 2
17 13 12 11 8 7 6 5 3 2
```

Response

13 8 7 6 5 17 12 11 3 2

Exercise 2 (1.5 points)

Consider a binary tree whose visits return the following sequences.

```
Pre-order:      A   B   E   C   L   D   F   G   H   I
In-order:       E   B   L   C   A   F   D   H   G   I
Post-order:      E   L   C   B   F   H   I   G   D   A
```

Report the sequence of keys stored on the tree leaves, moving on the tree from left to right. Please, report the list of keys on the same line, separated by a single space. No other symbols must be included in the response. This is an example of the response format: A X C Y etc.

Solution

```
### Tree guess
Node Key Left Right
1      A      2      3
2      B      4      5
3      D      6      7
4      E NULL NULL
5      C      8 NULL
6      F NULL NULL
7      G      9     10
8      L NULL NULL
9      H NULL NULL
10     I NULL NULL
```

Response

E L F H I

Exercise 3 (4.0 points)

Suppose a BST stores strings of unknown size. Each BST node has a standard structure but it also includes an integer value storing the size of the BST rooted at that node. Write the function to insert a new key into the leaves of the BST. The node type and the function prototypes are the following.

```
typedef struct node_s {
    char *key;
    int size;
    struct node_s *left;
    struct node_s *right;
} node_t;
node_t *insert (node_t *root, char *key);
```

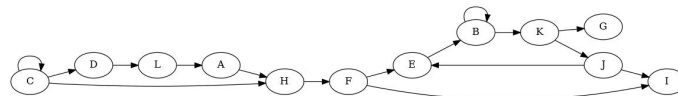
Pay particular attention to how to update the size of all visited nodes.

Solution

Please, see the teacher's overhead for a complete analysis of the topic.

Exercise 4 (1.5 points)

Visit the following graph in breadth-first, starting at node C. When necessary, consider nodes and edges in alphabetic order.



Display the minimum distance of all vertices from the starting one. Please, indicate the distance of all vertices sorted in alphabetic order (i.e., display the distance for A B C D, etc.). Report a sequence of integer values separated by one single space. No other symbols must be included in the response. This is an example of the response: 0 3 2 6 8 etc.

Solution

```

Directed matrix ... stated as such.
UN-weighted matrix ...stated as such.
Vertex list: [ 0] A [ 1] B [ 2] C [ 3] D [ 4] E [ 5] F [ 6] G [ 7] H [ 8] I [ 9] J [10] K [11] L
Adjacency Matrix:
0 0 0 0 0 0 0 0 1 0 0 0 0
0 1 0 0 0 0 0 0 0 0 0 0 1
0 0 1 1 0 0 0 0 1 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 1
0 1 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 1 0 0 0 0 1 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 1 0 0 0 0 1 0 0 0
0 0 0 0 0 0 1 0 0 0 1 0 0
1 0 0 0 0 0 0 0 0 0 0 0 0
### Graph visit
BFS:
Node=[ 0] A Discovery_time= 3 Predecessor=[11]
Node=[ 1] B Discovery_time= 4 Predecessor=[ 4]
Node=[ 2] C Discovery_time= 0 Predecessor=[-1]
Node=[ 3] D Discovery_time= 1 Predecessor=[ 2]
Node=[ 4] E Discovery_time= 3 Predecessor=[ 5]
Node=[ 5] F Discovery_time= 2 Predecessor=[ 7]
Node=[ 6] G Discovery_time= 6 Predecessor=[10]
Node=[ 7] H Discovery_time= 1 Predecessor=[ 2]
Node=[ 8] I Discovery_time= 3 Predecessor=[ 5]
Node=[ 9] J Discovery_time= 6 Predecessor=[10]
Node=[10] K Discovery_time= 5 Predecessor=[ 1]
Node=[11] L Discovery_time= 2 Predecessor=[ 3]
DFS:
List of edges:
[ 2] C -> [ 2] C : (B) Back
[ 2] C -> [ 3] D : (T) Tree
[ 3] D -> [11] L : (T) Tree
[11] L -> [ 0] A : (T) Tree
[ 0] A -> [ 7] H : (T) Tree
[ 7] H -> [ 5] F : (T) Tree
[ 5] F -> [ 4] E : (T) Tree
[ 4] E -> [ 1] B : (T) Tree
[ 1] B -> [ 1] B : (B) Back
[ 1] B -> [10] K : (T) Tree
[10] K -> [ 6] G : (T) Tree
[10] K -> [ 9] J : (T) Tree
```

```

[ 9] J -> [ 4] E : (B) Back
[ 9] J -> [ 8] I : (T) Tree
[ 5] F -> [ 8] I : (F) Forward
[ 2] C -> [ 7] H : (F) Forward
List of vertices:
[ 0] A: dist_time= 4 endp_time=21 pred=[11] L
[ 1] B: dist_time= 8 endp_time=17 pred=[ 4] E
[ 2] C: dist_time= 1 endp_time=24 pred=[-1] -
[ 3] D: dist_time= 2 endp_time=23 pred=[ 2] C
[ 4] E: dist_time= 7 endp_time=18 pred=[ 5] F
[ 5] F: dist_time= 6 endp_time=19 pred=[ 7] H
[ 6] G: dist_time=10 endp_time=11 pred=[10] K
[ 7] H: dist_time= 5 endp_time=20 pred=[ 0] A
[ 8] I: dist_time=13 endp_time=14 pred=[ 9] J
[ 9] J: dist_time=12 endp_time=15 pred=[10] K
[10] K: dist_time= 9 endp_time=16 pred=[ 1] B
[11] L: dist_time= 3 endp_time=22 pred=[ 3] D
Reachable nodes from vertex [ 2] C:
[ 0] A - [ 1] B - [ 2] C - [ 3] D - [ 4] E - [ 5] F - [ 6] G - [ 7] H - [ 8] I - [ 9] J - [10] K - [11] L -
UN-reachable nodes from vertex [ 2] C:
none

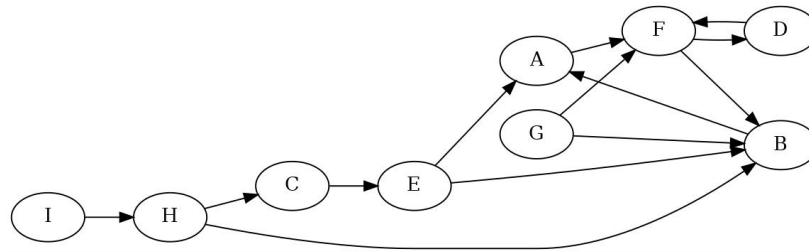
```

Response

3 4 0 1 3 2 6 1 3 6 5 2

Exercise 5 (1.5 points)

Visit the following graph in depth-first, starting at node I. Label each edge as tree (T), back (B), forward (F), and cross (C). When necessary, consider nodes and edges in alphabetic order.



Report the label of the edges CE, GB, and GF in this order. Please, indicate the edge type of these 3 arcs with a single letter, i.e., T, F, B, C, separated by single spaces. No other symbols must be included in the response. This is an example of the response: F T B

Solution

```

Directed matrix ... stated as such.
UN-weighted matrix ...stated as such.
Vertex list: [ 0] A [ 1] B [ 2] C [ 3] D [ 4] E [ 5] F [ 6] G [ 7] H [ 8] I
Adjacency Matrix:
0 0 0 0 0 1 0 0 0
1 0 0 0 0 0 0 0 0
0 0 0 0 1 0 0 0 0
0 0 0 0 0 1 0 0 0
1 1 0 0 0 0 0 0 0
0 1 0 1 0 0 0 0 0
0 1 0 0 0 1 0 0 0
0 1 1 0 0 0 0 0 0
0 0 0 0 0 0 0 1 0
### Graph visit
BFS:
Node=[ 0] A Discovery_time= 3 Predecessor=[ 1]
Node=[ 1] B Discovery_time= 2 Predecessor=[ 7]
Node=[ 2] C Discovery_time= 2 Predecessor=[ 7]
Node=[ 3] D Discovery_time= 5 Predecessor=[ 5]
Node=[ 4] E Discovery_time= 3 Predecessor=[ 2]
Node=[ 5] F Discovery_time= 4 Predecessor=[ 0]
Node=[ 6] G Discovery_time= - Predecessor=[-1]
Node=[ 7] H Discovery_time= 1 Predecessor=[ 8]

```

```

Node=[ 8] I Discovery_time= 0 Predecessor=[-1]
DFS:
List of edges:
[ 8] I -> [ 7] H : (T) Tree
[ 7] H -> [ 1] B : (T) Tree
[ 1] B -> [ 0] A : (T) Tree
[ 0] A -> [ 5] F : (T) Tree
[ 5] F -> [ 1] B : (B) Back
[ 5] F -> [ 3] D : (T) Tree
[ 3] D -> [ 5] F : (B) Back
[ 7] H -> [ 2] C : (T) Tree
[ 2] C -> [ 4] E : (T) Tree
[ 4] E -> [ 0] A : (C) Cross
[ 4] E -> [ 1] B : (C) Cross
[ 6] G -> [ 1] B : (C) Cross
[ 6] G -> [ 5] F : (C) Cross
List of vertices:
[ 0] A: dist_time= 4 endp_time= 9 pred=[ 1] B
[ 1] B: dist_time= 3 endp_time=10 pred=[ 7] H
[ 2] C: dist_time=11 endp_time=14 pred=[ 7] H
[ 3] D: dist_time= 6 endp_time= 7 pred=[ 5] F
[ 4] E: dist_time=12 endp_time=13 pred=[ 2] C
[ 5] F: dist_time= 5 endp_time= 8 pred=[ 0] A
[ 6] G: dist_time=17 endp_time=18 pred=[-1] -
[ 7] H: dist_time= 2 endp_time=15 pred=[ 8] I
[ 8] I: dist_time= 1 endp_time=16 pred=[-1] -
Reachable nodes from vertex [ 8] I:
[ 0] A - [ 1] B - [ 2] C - [ 3] D - [ 4] E - [ 5] F - [ 6] G - [ 7] H - [ 8] I -
UN-reachable nodes from vertex [ 8] I:
none

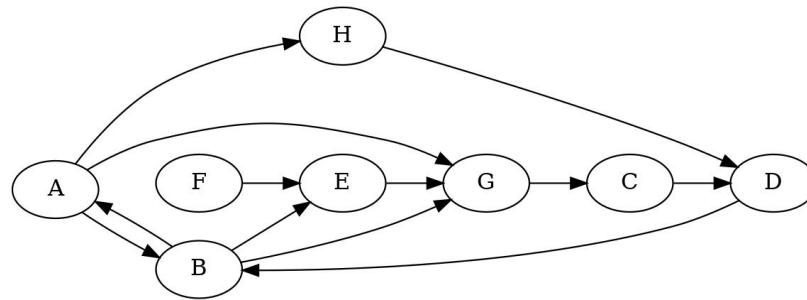
```

Response

T C C

Exercise 6 (1.5 points)

Given the following directed graph, represent the reverse graph and find all strongly connected components.



Compute all strongly connected components in the graph. Report them in alphabetic order and within each component indicates vertices in alphabetic order (i.e., ACZ and not CAZ or ZAC). Separate components with a single space. No other symbols must be included in the response. This is an example of the response format: ABC EF XY etc.

Solution

```

Directed matrix ... stated as such.
UN-weighted matrix ...stated as such.
Vertex list: [ 0] A [ 1] B [ 2] C [ 3] D [ 4] E [ 5] F [ 6] G [ 7] H
Adjacency Matrix:
0 1 0 0 0 0 1 1
1 0 0 0 1 0 1 0
0 0 0 1 0 0 0 0
0 1 0 0 0 0 0 0
0 0 0 0 0 0 1 0
0 0 0 0 1 0 0 0
0 0 1 0 0 0 0 0
0 0 0 1 0 0 0 0
### Graph SCC

```

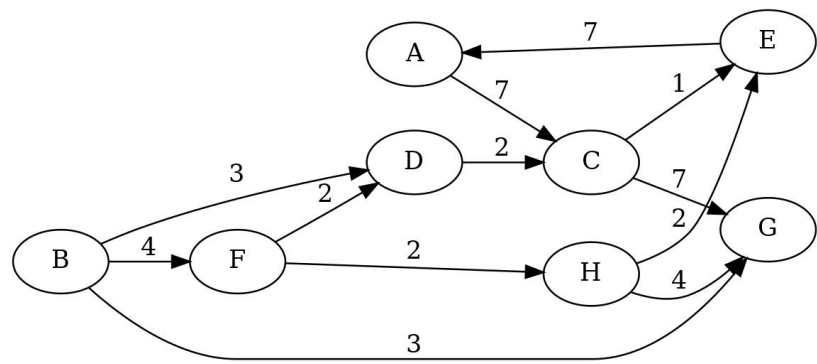
```

Visit on G^T:
[ 0] A: dist_time= 1 endp_time=16
[ 1] B: dist_time= 2 endp_time=15
[ 2] C: dist_time= 4 endp_time=11
[ 3] D: dist_time= 3 endp_time=14
[ 4] E: dist_time= 6 endp_time= 9
[ 5] F: dist_time= 7 endp_time= 8
[ 6] G: dist_time= 5 endp_time=10
[ 7] H: dist_time=12 endp_time=13
Visit on G:
[ 0] A: dist_time= 1 endp_time=14
[ 1] B: dist_time= 2 endp_time=11
[ 2] C: dist_time= 5 endp_time= 8
[ 3] D: dist_time= 6 endp_time= 7
[ 4] E: dist_time= 3 endp_time=10
[ 5] F: dist_time=15 endp_time=16
[ 6] G: dist_time= 4 endp_time= 9
[ 7] H: dist_time=12 endp_time=13
Number of SCCs: 2
SCC0: [ 0] A [ 1] B [ 2] C [ 3] D [ 4] E [ 6] G [ 7] H
SCC1: [ 5] F

```

Response
 ABCDEGH F

Exercise 7 (1.5 points)
 Given the following directed and weighted graph, apply Dijkstra’s algorithm to find all shortest paths connecting node B with all the other nodes. When necessary, consider nodes and edges in alphabetical order.



Report the shortest paths to all vertices. Please, indicate the shortest path to all vertices sorted in alphabetic order (i.e, display the shortest path for A B C D etc.). Report a sequence of integer values separated by one single space. No other symbols must be included in the response. This is an example of the response: 0 3 2 6 8 etc.

Solution
 Directed matrix ... stated as such.
 Weighted matrix ... stated as such.
 Vertex list: [0] A [1] B [2] C [3] D [4] E [5] F [6] G [7] H
 Adjacency Matrix:

```

0 0 7 0 0 0 0 0
0 0 0 3 0 4 3 0
0 0 0 0 1 0 7 0
0 0 2 0 0 0 0 0
7 0 0 0 0 0 0 0
0 0 0 2 0 0 0 2
0 0 0 0 0 0 0 0
0 0 0 0 2 0 4 0

```

 ### Graph SSSP (Dijkstra and Bellman-Ford)
 Dijkstra
 Shortest paths from vertex [1] B
 Node [1] B: 0 (pred=-1)
 Node [3] D: 3 (pred= 1)

```

Node [ 6] G: 3 (pred= 1)
Node [ 5] F: 4 (pred= 1)
Node [ 2] C: 5 (pred= 3)
Node [ 4] E: 6 (pred= 2)
Node [ 7] H: 6 (pred= 5)
Node [ 0] A: 13 (pred= 4)
Bellman Ford
Iteration 0: A:infy B:0 C:5 D:3 E:8 F:4 G:3 H:6
Iteration 1: A:13 B:0 C:5 D:3 E:6 F:4 G:3 H:6
Iteration 2: A:13 B:0 C:5 D:3 E:6 F:4 G:3 H:6
Shortest paths from vertex [ 1] B
Node [ 0] A: 13 (pred= 4)
Node [ 1] B: 0 (pred=-1)
Node [ 2] C: 5 (pred= 3)
Node [ 3] D: 3 (pred= 1)
Node [ 4] E: 6 (pred= 2)
Node [ 5] F: 4 (pred= 1)
Node [ 6] G: 3 (pred= 1)
Node [ 7] H: 6 (pred= 5)

```

Response

13 0 5 3 6 4 3 6

Exercise 8 (2.0 points)

Analyze the following program and indicate the exact output it generates. Please, report the exact program output with no extra symbols.

```

#define MAX 32
void f (char s1[], char s2[], char s3[]);
int main(void) {
    char s1[] = "010101";
    char s2[] = "011101";
    char s3[] = "xxxxxx";
    f (s1, s2, s3);
    printf ("%s", s3);
    return (1);
}
void f (char s1[], char s2[], char s3[]) {
    int i, a, b, c, s, l;
    l = strlen(s1) > strlen(s2) ? strlen(s1) : strlen(s2);
    s3[l] = '\0';
    c = 0;
    for (i=l-1; i>=0; i--) {
        a = s1[i] - '0';
        b = s2[i] - '0';
        s = a + b + c;
        s3[i] = s%2 + '0';
        c = s/2;
    }
    return;
}

```

Response

110010

Exercise 9 (2.0 points)

The following function stores two files into a list of lists.

```

typedef struct m m_t;
typedef struct p p_t;
struct m {
    char *name;
    char *id;
    p_t *p;
    m_t *next;
};
struct p {
    char *name;
    int price;
    p_t *next;
};

```

```

m_t *load (char *filename1, char *filename2) {
    m_t *head, *new1;
    p_t *new2;
    FILE *fp;
    int price, found;
    char str1[MAX], str2[MAX];
    head = NULL;
    fp = fopen (filename1, "r");
    while (fscanf (fp, "%s%s", str1, str2) != EOF) {
        new1 = (m_t *) malloc (sizeof (m_t));
        if (new1==NULL) {
            fprintf (stdout, "Allocation error.\n");
            exit (1);
        }
        new1->name = strdup (str1);
        new1->id = strdup (str2);
        new1->p = NULL;
        new1->next = head;
        head = new1;
    }
    fclose(fp);
    fp = fopen (filename2, "r");
    while (fscanf (fp, "%s%s%d", str1, str2, &price) != EOF) {
        new2 = (p_t *) malloc (sizeof (p_t));
        if (new2==NULL) {
            fprintf (stdout, "Allocation error.\n");
            exit (1);
        }
        new2->name = strdup (str2);
        new2->price = price;
        found = 0;
        new1 = head;
        while (new1!=NULL && found==0) {
            if (strcmp (new1->id, str1)==0) {
                found = 1;
            } else {
                new1 = new1->next;
            }
        }
        if (found==1) {
            new2->next = new1->p;
            new1->p = new2;
        } else {
            fprintf (stderr, "There is no such a brand ... Rule-out model\n");
        }
    }
    fclose(fp);
    return head;
}

```

Indicates which ones of the following statements are correct. Note that incorrect answers imply a penalty in the final score.

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.

Response

- 1.
- 2.
- 3.
- 4.

5.

6.

Exercise 10 (2.0 points)

Analyze the following program and indicate the exact output it generates. Please, report the exact program output with no extra symbols.

```
void f (int i, int j, int k) {
    if (i<0 && j<0 && k<0) {
        return;
    }
    if (i>=0) {
        printf ("I");
        f (i-1, j, k);
    } else {
        if (j>=0) {
            printf ("J");
            f (i, j-1, k);
        } else {
            if (k>=0) {
                printf ("K");
                f (i, j, k-1);
            }
        }
    }
    return;
}

int main () {
    f(5,4,3);
    return 1;
}
```

Response

IIIIIJJJJJKKK

Exercise 11 (3.0 points)

A square matrix m of size n stores integer values.

Write the function

```
void check (int **m, int n, int k);
```

which receives the matrix m , its size n , and a value k (smaller than n) and it displays the submatrix of m of size k whose sum of the element is the largest, and the submatrix of m of size k whose sum of the element is the smallest.

For example, if $n = 6$, $k = 3$, and the matrix m is the following one

```
0 1 0 1 0 1
1 0 1 0 1 0
0 1 0 0 0 1
1 0 1 0 1 0
0 1 0 0 0 0
3 0 1 0 2 0
```

the submatrix of size three with the largest sum is

```
1 0 1
0 1 0
3 0 1
```

and the one with the smallest sum is


```
0 0 1
0 1 0
0 0 0
```

Write the entire program using standard C libraries but implement all required personal libraries. Modularize the program adequately, and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward programs, complex or impossible to understand, will be penalized in terms of the final evaluation.

Solution

To be done.

Exercise 12 (5.0 points)

A file includes an undefined number of rows. Each row includes two fields: A string of at most 100 characters, and an integer value. Those fields are separated by one or more spaces.

Write a C function to organize the file content into a list of lists. The elements of the main list represent different strings (i.e., strings are used as the main key of the data structure). The elements of the secondary lists represent different integers (i.e., integers are adopted as the secondary key of the data structure). More textually:

- - All records of the file with the same string must be stored in the same secondary list.
- All records of the file with the same string and the same integer value must be stored in the same element of a secondary list. Each element of the secondary list also stores the number of rows with the same string and integer in the file.

Write the function:

```
list1_t insert (char *name);
```

which receives the file name and returns the pointer of such a list.

Define the type `list1_t` and `list2_t` for the primary and secondary lists. Lists do not have to be sorted. The string within the `list1_t` type must be dynamically allocated.

For example, if we suppose that the file has the following content

```
ALFONSO 13
MARIA 15
CLARA 12
RAFFAELA 33
ALFONSO 12
CLARA 12
RAFFAELA 33
CLARA 12
```

the data structure created by the function must be the following one.

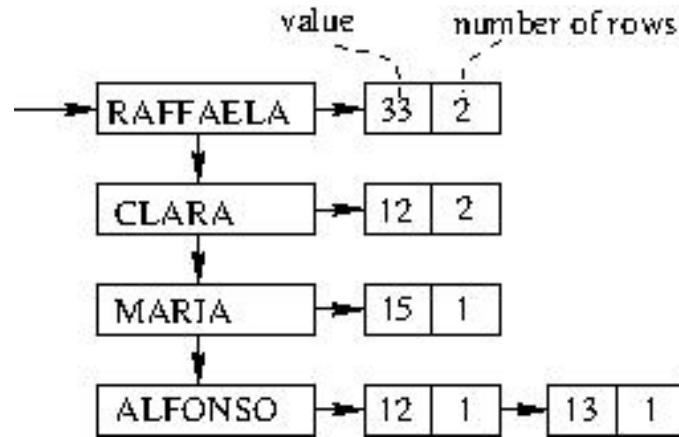
Write the entire program using standard C libraries but implement all required personal libraries. Modularize the program adequately, and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward programs, complex or impossible to understand, will be penalized in terms of the final evaluation.

Solution

To be done.

Exercise 13 (9.0 points)

A basic calculator supports the four basic operations on integer decimal numbers: Division, multiplication, addition, and subtraction. All operations can be applied only to integer numbers and return an integer number (including the



division, i.e., $11/5 = 2$). Moreover, all operators have the same precedence and they are left-associative. For example, the expression $22 - 21 * 7/3$ must be evaluated as $((22 - 21) * 7)/3 = 2$.

Write the function

```
void calculator (int *v, int n, int result);
```

which receives an array v of size n of integer values, and the integer value $result$, and it displays all possible expressions delivering the result.

For example, if $v = \{3, 2, 25, 5\}$, that includes four integer values (i.e., $n=4$), and the result of the expression is 5 (i.e., $result=5$), the expressions leading to that value (and that the function must display) are $(2 * 3 * 5 - 25)$, $(3 * 2 * 5 - 25)$, $(3 * 5 * 2 - 25)$, etc., $(25/3 - 5 + 2)$, $(25/3 + 2 - 5)$, etc.

Solution

To be done.